AI-Driven Robot-Building Game Architecture

https://share.gemini.google/cwBrEPWNP3rq

Created with **3.8 Thinking** October 25, 2026 at 10:41 AM • Published October 25, 2026 at 12:29 PM

I'm trying to set up a game using Gemini that will have different levels, and the levels will be spawned by just generative AI spawned as the game progresses in real time. Maybe that's not the idea, but that's what I have ri

The core idea is not bad at all—in fact, a game that bridges digital gameplay directly into a physical, working 3D-printed robot is a compelling "holy grail" for educational tech and maker culture.

However, trying to generate both arbitrary real-time 3D game levels and unconstrained, mechanically functional 3D CAD models entirely on the fly with generative AI will break under current technical limits. To make this work smoothly, you need to turn unconstrained generation into **guided modular generation**.

Here is an architectural breakdown of what works, what fails, and how to execute it realistically.

Continue this chat

1. Where the Pure "Generative" Id

Gemini may display inaccurate info, including about people, so double-check its responses.

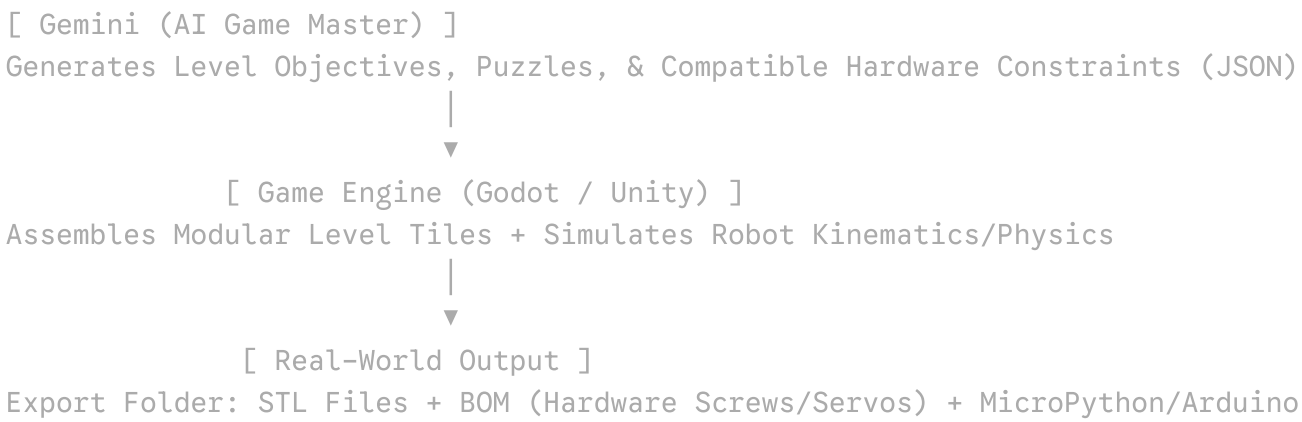
Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

robot generated from pure text prompts will lack functional gear meshing, motor mounting patterns, wire routing, structural rigidity, and fastener tolerances (like M3 heat-set inserts). It won't work in the physical world without human redesign.

Level Latency and Physics Instability: Having an LLM generate raw 3D level geometry in real time creates massive latency spikes between rooms, broken collision hulls, and unplayable physics bugs.

2. The Solution: A Hybrid "Modular Kit" Architecture

Instead of having Gemini generate arbitrary 3D models out of thin air, treat Gemini as a **Procedural Director and Systems Integrator** operating over a curated, real-world hardware library.



A. The Robot System: Open-Source Modular Primitives

Standardize the chassis and mounting interfaces around real, off-the-shelf components.

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Off-the-Shelf Electronics (COTS): Lock in standard, cheap components:

Actuators: SG90/MG996R hobby servos, NEMA 17 steppers, or standard DC gearmotors.

Brain: ESP32 or Raspberry Pi Pico.

Power: 18650 battery sleds or USB power banks.

How Generative AI Fits In:
Gemini doesn't model the CAD from scratch; it handles **parametric customization and aesthetic skinning** (e.g., adjusting arm link lengths within safe torque envelopes, or generating stylized aesthetic armor/grippers around fixed mechanical hardpoints).

B. The Level System: Semantic Generation (JSON to Tile Engine)

Instead of asking Gemini to output 3D geometry directly, have it output structured JSON directives to your game engine (Godot or Unity).

Gemini acts as a dynamic level designer:

JSON

```
{
  "room_theme": "Industrial Sorting Bay",
  "required_capability": "2-DOF manipulation + color sensing",
  "environmental_hazard": "low_clearance_gap_40cm",
  "reward_part": {
    "id": "end_effector_gripper_v2",
    "compatible_motor": "SG90_servo",
    "stl_manifest_id": "stl_claw_assembly_04"
  }
}
```

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

The game engine receives this JSON and instantly places pre-baked, physics-tested modular floor tiles, obstacles, and objective triggers. This keeps level generation sub-second and mechanically bug-free.

3. The Core Gameplay Loop

1. **Chassis Selection:** The player selects an archetype:

   Differential Drive AV (ground rover)

   SCARA or 3-Axis Industrial Arm (fixed base, precision)

   Biped/Humanoid or Quadruped (high complexity, terrain adaptation)

2. **The Progression Gate:** Each level presents an environmental puzzle that demands a physical upgrade.

   Example: A level requires reaching an elevated switch across a gap. A standard rover chassis cannot reach it.

   The AI GM: Generates an objective to retrieve an actuator mount.

   The Assembly Screen: The player equips the newly unlocked joint in-game. The game runs an in-engine kinematic solver (e.g., checking torque limits and center of mass) to verify it can physically move the arm without tipping over.

3. **The Payoff (Endgame Export):**
   When the player beats the final level, the game generates a complete build package:

   ZIP File of Print-Ready STLs: Pre-oriented for FDM 3D printing (no crazy overhangs).

   Bill of Materials (BOM): A clean checklist with exact screw lengths, wire gauges, and direct links to standard parts (Amazon/AliExpress).

   Firmware: Auto-generated MicroPython or Arduino code containing the exact pin mapping and inverse kinematics calibrated to their in-game configuration.

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Building humanoids, cobots, and rovers all at once will stall the project.

Best starting prototype: An **Autonomous Rover (AV) with a 2-to-3 Axis Arm**.

Ground vehicles have forgiving centers of gravity.

3D-printed wheels, chassis brackets, and micro-servo arms are cheap, reliable, and mechanically forgiving to assemble in real life.

Bipeds/humanoids require active balance algorithms, high-torque bus servos, and strict mass distribution that easily frustrate end-users trying to build them physically.

Which game engine are you planning to build this in, and do you already have a 3D printing/hardware background?

We're planning on using Rust just for speed and with a desktop app. It's not a web app, it's a desktop app, and it should have its dependencies. We need to use acceleration, like MPS.

Targeting a native Rust desktop application with hardware acceleration—specifically leveraging Apple Silicon's Metal Performance Shaders (MPS) and unified memory—is an ideal architecture for keeping simulation physics, rendering, and generative pipelines responsive.

Because Gemini operates as a cloud-hosted API, your architecture splits naturally: **Gemini acts as the remote Game Master (strategic level design, puzzle generation, and dynamic narrative)**, while **Rust handles native graphics, rigid-body physics, kinematic simulation, and optional local neural compute via Metal/MPS.**

The Recommended Rust Technology Stack

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Domain	Recommended Crate / Tool	Why It Fits Your Requirements
Game Engine & Rendering	`bevy` (using `wgpu`)	High-performance data-driven ECS (Entity Component System). `wgpu` automatically targets the **Metal** backend on macOS, giving native hardware acceleration on Apple Silicon GPUs.
Physics Simulation	`bevy_rapier3d` or `avian3d`	Written in pure Rust. Handles rigid-body dynamics, articulated joints (revolute, prismatic, spherical), friction, and collision meshes essential for robot balance.
Robotics Kinematics	`k` or `urdf-rs`	Industrial-grade kinematics libraries in Rust. Calculates Forward Kinematics (FK) and Inverse Kinematics (IK) to verify that a robot configuration can physically reach puzzle targets.
Local Neural Compute (MPS)	`candle-core` (with Metal feature)	Hugging Face's minimalist ML framework for Rust. If you run local models (e.g., local terrain generation, physics surrogates, or small SLMs), `candle` uses

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Domain	Recommended Crate / Tool	Why It Fits Your Requirements
Async Networking	`tokio` + `reqwest`	Handles background Gemini REST/Streaming API calls so level generation never hitches the main 60/120 FPS game loop.
Mesh & CAD Export	`stl_io` / `truck`	Generates, validates, and packages watertight manifold STL meshes for real-world slicing and 3D printing.

The Secret Weapon: Build Around URDF

Instead of inventing a custom format to represent your robot, use **URDF (Unified Robot Description Format)**.

URDF is an XML standard maintained by the ROS (Robot Operating System) community that defines:

1. **Links:** The physical rigid bodies (mass, center of inertia, visual 3D mesh, collision hull).
2. **Joints:** How links connect (rotational limits, maximum torque, axis of motion).

Why this matters for your game:

When Gemini suggests a robot configuration, the game engine assembles it as a URDF tree in memory.

Rust's `k` crate can parse the URDF, compute joint limits, and calculate whether the motors have enough torque to lift the arm without tipping the robot over.

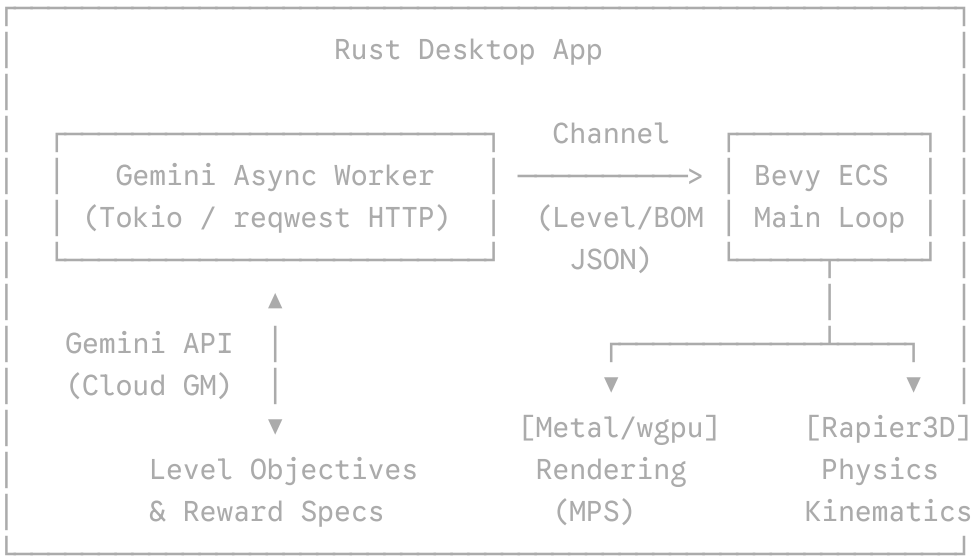
Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Runtime Architecture: Decoupling the Game Loop from Gemini

Gemini API calls will take between 400ms to 2000ms. In Rust, you must isolate the generative loop from the render/physics loop using Bevy's asynchronous task pool.



1. The Async Task (Background Gemini Worker)

Bevy's `AsyncComputeTaskPool` spawns an asynchronous worker that sends a structured schema request to Gemini (using JSON mode) whenever a player approaches a level transition:

Rust

Continue this chat

```rust
use serde::{Deserialize, Serialize};
```

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

```
    pub biome_id: String,
    pub obstacle_type: String,
    pub target_reach_height_cm: f32,
    pub unlocked_part: RobotPartDefinition,
}

#[derive(Serialize, Deserialize, Debug)]
pub struct RobotPartDefinition {
    pub part_id: String,
    pub mesh_asset_path: String,
    pub mount_type: String, // e.g., "M3_faceplate_40mm"
    pub mass_grams: f32,
    pub torque_capacity_kg_cm: Option<f32>,
}
```

2. The Verification Gate (Physics / Kinematics Check)

When the level unlocks a new 3D-printable bracket or motor mount:

1. The player snaps the part onto their chassis in the assembly UI.
2. The Rust physics system runs a dynamic equilibrium check:

   Mass Distribution: Calculates the combined center of mass (CoM). If the arm extends and the CoM moves outside the polygon of the rover's wheels or humanoid's foot base, the robot falls over.

   Torque Validation: Evaluates $\tau = r \times F$. If the servo's rated torque (e.g., standard SG90 servo = $1.8\ \text{kg} \cdot \text{cm}$) is less than the lever arm load, the joint refuses to actuate.
3. The player must redesign or balance their chassis before the level puzzle can be cleared.

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

When the campaign concludes, your desktop app executes an export system using Rust's file I/O:

1. **Geometry Package (** `/exports/stls/` **):**

   Uses the pre-modeled, tolerance-tested STLs tied to the assembled parts.

   Each part includes pre-calibrated clearance holes (3.2mm for M3 pass-through, 4.0mm for M3 brass heat-set inserts).

2. **Bill of Materials (** `/exports/BOM.html` **or** `.csv` **):**

   Aggregates every non-printed part used: quantity of standard bearings (e.g., 608ZZ), fastener count (e.g., `14x M3x12mm socket head cap screws`), servo models, and microcontroller recommendations.

3. **Firmware Scaffold (** `/exports/firmware/main.py` **):**

   Since the game tracks joint zero-points and pin assignments, it emits a ready-to-flash MicroPython script configured to the exact geometric lengths and pinouts selected in-game.

Immediate Next Step to Prototype

Do not build the full level generator first. Build a minimal Bevy 3D scene that loads:

1. A static base rover chassis.
2. An articulated 2-link arm (
3. An async system that que

this needs to b optimized for me to listen. tabular data is hard to. read aloud

`"Pick up the battery at coordinates [0.5, 0.4, 0.1]`), applies the coordinates to an in-game target, and attempts to solve it using inverse kinematics.

Building this in Rust as a native desktop application lets you divide the workload cleanly: Gemini works in the cloud as your dynamic game director, while your local machine uses Apple Silicon

Opinion: this is yo

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

The Rust Technology Stack

Instead of an unwieldy tech stack, you can rely on five core Rust tools that work naturally with Apple Silicon:

For your game engine and rendering: Use **Bevy** paired with the **wgpu** graphics library. On macOS, wgpu automatically routes straight into Apple's native Metal backend. This gives you hardware-accelerated 3D graphics on your Mac's GPU right out of the box.

For rigid-body physics: Use **Bevy Rapier 3D** or **Avian 3D**. Both are written in pure Rust and handle real-world collisions, friction, and mechanical joints like hinges and slides.

For robot kinematics: Use the **k** crate or **urdf-rs**. These are industrial-grade robotics libraries that calculate forward and inverse kinematics, confirming whether a robot arm can physically reach its target.

For local neural network acceleration: If you want to run small helper models or procedural terrain tools locally, use Hugging Face's **Candle** framework with the Metal feature enabled. Candle talks directly to Apple's Metal Performance Shaders without dragging in any Python runtime overhead.

For networking: Use **Tokio** and **reqwest**. These handle background communication with the Gemini API so network traffic never hitches your 60-frame-per-second render loop.

The Secret Weapon: URDF

Rather than inventing a custom file format for your robots, build the entire game around **URDF**, which stands for the **Unified Robot Description Format**.

URDF is the open standard used across professional robotics. It breaks every machine down into two core concepts: **links**, which represent the physical rigid bodies and their 3D meshes, and **joints**, which represent the rotational or sliding limits connecting those bodies.

When Gemini invents a robot configuration to solve a puzzle, your Rust code builds a virtual robot tree in memory using URDF. The game can immediately check whether the real-world

Continue the quiz

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

player finishes the game, they do not just get loose 3D meshes; they get a standard robotics file they can inspect in professional simulators like ROS or Gazebo.

The Dynamic Game Loop

The gameplay balances procedural challenges with physical engineering checks in four continuous steps:

1. **The Challenge:** The player drives into a new sector and hits an obstacle they cannot clear with their current build, such as reaching an elevated power switch across a pit.
2. **The Asynchronous AI Director:** In the background, a worker thread sends the room context to Gemini. Gemini returns a structured data payload containing the puzzle layout and a compatible reward, like a two-finger gripper mount designed for a micro servo.
3. **The Assembly Screen:** The player snaps the newly unlocked 3D-printable part onto their chassis.
4. **The Physics Sanity Check:** Before allowing the player to proceed, the Rust engine runs real-world mechanics checks. It calculates the robot's center of mass; if an extended arm drags the center of mass outside the footprint of the wheels, the robot tips over. It also calculates torque: if the motor’s rated force cannot overcome the weight of the extended arm, the joint simply stalls. The player must reconfigure their parts until the design is mechanically sound.

The Physical Export Pipeline

When the campaign wraps up, your Rust application writes a ready-to-build manufacturing bundle directly to the player's desktop:

A folder of 3D printable files: Clean, watertight STL files pre-calibrated with standard clearances, such as 3.2-millimeter holes for standard M3 screws and 4-millimeter cavities for brass heat-set inserts.

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

holders, and microcontroller boards.

Pre-configured firmware: A ready-to-flash MicroPython or Arduino script tailored to the exact link lengths, joint limits, and pinouts the player selected in the game.

Starting with an autonomous wheeled rover with a two-axis arm is the safest first milestone, as ground rovers are far more forgiving with balance and motor power than two-legged humanoids.

Would you like to focus next on structuring the asynchronous background loop between Bevy and Gemini, or on how the physical joint constraints will be validated?

I need you to get updated with the latest models for Gemini, the latest Google models, and the latest agent technologies. Very important that you get updated. Do not give me old information. I need to know what the differences a

Google's AI and agent ecosystem centers on the **Gemini Enterprise Agent Platform** (an evolution and consolidation of Vertex AI and Vertex AI Agent Builder). It is structured around the **Gemini 3 generation** models and an enterprise governance stack designed around the Agent-to-Agent (A2A) protocol and Model Context Protocol (MCP). JonJones.AI + 1

1. Latest Google Gemini & Frontier Models

The active production lineup is led by the **Gemini 3 family**: Reddit

Gemini 3.8 Flash: The primary flagship general-purpose and agentic model. It provides deep multi-step reasoning, low latency, long-context tool invocation, and token-efficient generation. Lorka AI + 1

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

agents.

Gemini 3.5 Flash-Lite: High-throughput, cost-optimized tier designed for high-frequency micro-agent steps, classification, routing, and iterative reflection loops. Reddit

Gemini Flash Cyber: A domain-tuned model specialized for vulnerability analysis, AST evaluation, and security telemetry auditing in software agents. Reddit

2. The GCP Agent Ecosystem: 4-Pillar Architecture

Google Cloud organizes its agentic architecture under four primary pillars: **Build**, **Scale**, **Govern**, and **Optimize**. JonJones.AI

Build: Houses the Agent Development Kit (ADK), Agent Garden (pre-built templates), and native MCP (Model Context Protocol) tool integration. Tobira - Tobira.ai

Scale (Runtime): Managed infrastructure including **Agent Compute Engine** (Titanium IPU-backed hardware-isolated microVMs with auto-sleep/resume) and **Managed Agents API**. Tech Times

Govern: Contains **Agent Identity**, **Agent Registry**, **Agent Gateway**, and **Policies**. Tobira - Tobira.ai

Optimize: Observability, continuous trace logging, auto-raters, and synthetic user persona simulation.

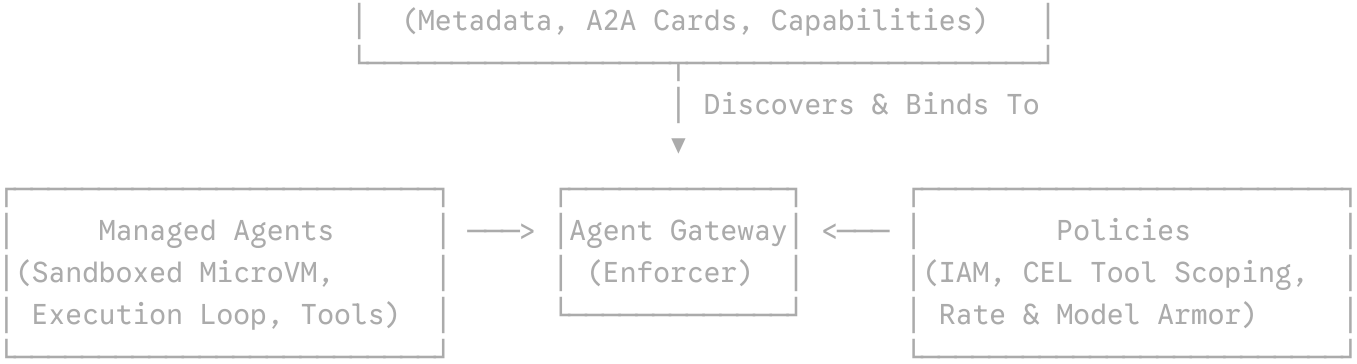
3. Comparing Agent Registry vs. Managed Agents vs. Policies

These components handle distinct responsibilities: **Agent Registry** is the catalog (Discovery/Metadata), **Managed Agents** is the execution environment (Runtime/Compute), and **Policies** are the rules and controls (Governance/Security). Tobira - Tobira.ai

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps



Detailed Breakdown

Dimension	Agent Registry	Managed Agents (API / Engine)	Policies (IAM Gateway)
Primary Role	**Discovery & Cataloging**	**Execution & Compute**	**Security & Gu**
What it represents	The central system of record for all verified agents, skills, and MCP servers across your enterprise tenant.	The autonomous compute harness that actually runs the agent loop and maintains environment state.	The determini dictating who to whom and actions are all
Key Mechanism	Stores standardized **A2A Agent Cards (JSON)**, versions, endpoints, and SPIFFE-backed Agent IDs.	Ephemeral or long-running **isolated microVM sandboxes**, local file system mounts, and	**IAM Allow/De** Common Expr Language (CE conditions, M Armor, and to

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Dimension	Agent Registry	Managed Agents (API / Engine)	Policies (IAM Gateway)
Lifecycle Moment	**Design / Registration Time:** When an agent is built and published for other systems to discover.	**Run Time (Task Execution):** When an agent plans, executes code, inspects data, and makes calls.	**Transaction T (Intercept):** E at the **Agent (** on every ingre egress tool/M
Managed By	Platform Administrators, Enterprise Governance teams.	Developers invoking the Managed Agents API or Agent Runtime.	SecOps, Clou Admins, and Compliance o

4. How They Interact

1. **Deploying the Worker (Managed Agents):**
   You configure a task using the *Managed Agents API* (or deploy custom code with ADK onto *Agent Compute Engine*). Google provisions a dedicated, isolated sandbox container with zero default internet/network access. Google Cloud Documenta...
2. **Publishing and Identity (Agent Registry):**
   Upon deployment, the agent receives an **Agent Identity**—a cryptographically verifiable SPIFFE identity backed by an X.509 certificate. The agent's schema, endpoints, and tool capabilities are indexed in the **Agent Registry** as an official *A2A (Agent-to-Agent)* entry. Other autonomous agents or internal systems query the Registry to discover what this agent can do. Tobira - Tobira.ai + 2
3. **Governing Communications (Policies):**

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

request and verifies the caller against configured **Policies**: Tobira - Tobira.ai

Tool Filtering via CEL: e.g., allowing read-only methods (`[iap.googleapis.com/mcp.tool.isReadOnly] (https://iap.googleapis.com/mcp.tool.isReadOnly) == true`) while rejecting destructive file overwrites.

Runtime Guardrails: Running Google *Model Armor* checks to scrub PII and neutralize prompt injection attempts. Tobira - Tobira.ai

Financial Guardrails: Enforcing strict token budget constraints before the model call proceeds. SiliconANGLE

This division allows developers to deplo[illegible] **Managed Agents** without circumven[illegible] can managed agents download files to your local mach granular control via **Policies** over an au[illegible]

Tobira - Tobira.ai

No, a managed agent running in Google Cloud or the Gemini API cannot directly push or write files onto your local machine.

Managed agents run inside an isolated, remote cloud sandbox (ephemeral Linux microVM) with zero ambient network access to your local workstation. However, **your local application can pull/download the files the agent creates.** Google + 1

The transfer of generated assets from a cloud sandbox to your local machine follows three primary patterns:

1. Direct Download via the Environments / Files API

When an agent generates files (code, CAD files, datasets, or reports), they reside in the remote environment's filesystem. Your local client application retrieves them using the `environment_id` : Google AI for Developers

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

Workspace archive (.tar): You pull an archive containing the entire `/workspace` directory: Google AI for Developers

Python

```python
import os
import requests
import tarfile

env_id = interaction.environment_id
api_key = os.environ["GEMINI_API_KEY"]

# Download the environment's snapshot
url = f"https://generativelanguage.googleapis.com/v1beta/files/environment-{env_id}:d
response = requests.get(
    url,
    params={"alt": "media"},
    headers={"x-goog-api-key": api_key},
    allow_redirects=True,
)

with open("output.tar", "wb") as f:
    f.write(response.content)

# Extract files to your local machine
with tarfile.open("output.tar") as tar:
    tar.extractall(path="./local_workspace")
```

2. Export via Cloud Storage (GCS) Continue this chat

For enterprise pipelines on GCP, the managed agent sandbox is given read/write mounts to a

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps

1. The agent writes its artifacts directly to a mounted GCS directory.

   Google Cloud Documenta...

2. Your local desktop app (or a Rust background worker) syncs from the bucket using the Cloud Storage API or a presigned URL.

3. Local Model Context Protocol (MCP) Bridge

If you need real-time, bidirectional file transfer while the agent runs:

You run a local MCP server on your machine that exposes explicit file-system tools (e.g., `write_local_file(path, content)`).

You expose the local server securely to the agent via a secure tunnel or Agent Gateway.

When the agent decides to save a file, it calls your local MCP tool to write the bytes directly to your disk.

Continue this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Google Privacy Policy Google Terms of Service Your privacy & Gemini Apps